

浙江大学实验报告

姓名：_____ 专业： 信息工程 _____ 学号： _____
课程名称： 人工智能 _____ 任课老师： _____
实验名称： 基于对抗式 AI 的五子棋实现 _____ 组号及组员： _____

1 选题背景

五子棋 (Gomoku) 作为经典的零和完全信息博弈游戏，其标准 15×15 棋盘的初始分支因子 b 高达 225。若采用 Minimax 算法，算力开销随搜索深度 (Depth) 呈 $O(b^d)$ 的指数级爆炸，因此需要引入 Alpha-Beta 剪枝。

本实验的底层主开发语言为 Python。不同于 C/C++ 能直接在物理内存和寄存器上操作，Python 作为动态类型的解释型语言，在单核 CPU 环境下评估一个盘面节点的极限速度约为 5,000 ~ 10,000 节点/秒。如果不加算力干预，面对 $O(225^d)$ 的搜索树，系统在实战要求的合理时间 (约 5 ~ 10 秒) 内连 Depth = 3 都难以流畅完成。如何在解释型语言的性能约束下，通过算法设计提升 Depth 使 AI 更聪明，是本课题面临的最大挑战。

2 设计目的

本设计旨在构建一个具备较高水平实战能力且具有基本交互体验的五子棋 AI 系统。具体工程目的如下：

- 综合运用动态分支收束、Zobrist 哈希、置换表、迭代加深与 MTD(f) 零宽探测算法，将 Python 语言下的搜索深度强行推升至 6 层；
- 使用 C++ 重写 MTD(f) 作为对照组，展示编译型语言在博弈算法中的性能优势 (Depth ≥ 8)；
- 在不依赖任何外部图片资产的前提下，利用代码渲染实现 3D 光影，并提供完善的人机工程交互体验；

3 算法原理

本系统的核心由 View 层 (处理图形界面)、Controller 层 (处理循环、输入输出的轮询与状态转换) 和 Agent 层 (处理落子方案) 构成。

其中，Agent 层分为两类实现方案。

3.1. Alpha-Beta 剪枝算法

为应对状态空间的指数级扩张，算法架构采用分层漏斗结构与迭代剪枝的策略。

3.1.1. Minimax 与 Alpha-Beta 剪枝机制

(1) 算法工作机理

在零和博弈中，系统基于极大极小树进行前向状态推演。定义局部启发式评估函数 $E(s)$ ，在深度为 d 的搜索树中：

Max 节点：模拟己方最优决策，选取子节点中 $E(s)$ 的最大值。

Min 节点：模拟最优对手决策，选取子节点中 $E(s)$ 的最小值。

为避免穷举 $O(b^d)$ 的全部叶子节点，引入 Alpha-Beta 剪枝。算法在深度优先搜索 (DFS)

中动态维护并向下传递最优解边界 $[\alpha, \beta]$:

α 记录当前 Max 节点所能确保的收益下界。

β 记录当前 Min 节点所能确保的收益上界。

(2) 截断条件

若在某子树的遍历过程中, 其评估值更新导致 $\beta \leq \alpha$, 则在数学上证明了理性对手在未来状态空间中存在更优的惩罚策略, 使得该分支永远无法作为纳什均衡的决策链被选中。系统将立刻中断该分支的拓展。

(3) 算法收益

将最坏情况下的时间复杂度从 $O(b^d)$ 优化至最佳情况的 $O(b^{\frac{d}{2}})$, 是使得有限时间内推演深层博弈树成为可能。

3.1.2. 矩阵视窗滑动特征提取

当搜索到达预设的最大深度时, 由于计算资源限制无法穷举至绝对终局, 算法必须调用启发式函数对当前的非终结节点进行静态估值。本系统采用矩阵视窗滑动机制对离散棋盘进行空间特征提取。

(1) 算法工作机理

定义一个一维空间长度 $L=5$ 的滑动特征核 (Kernel), 沿棋盘的基本正交与对角线方向特征向量 $\vec{d} \in \{(1,0), (0,1), (1,1), (1,-1)\}$ 遍历整个 15×15 离散矩阵。

针对特征核每次截取的状态子集, 统计其中的实体映射数量 (N_{self}, N_{op}):

若窗口为纯净状态 (即仅含一方实体, 如 $N_{self} > 0 \cap N_{op} = 0$), 该空间向量具备连成 5 子的拓扑潜能, 系统根据 N_{self} 赋予非线性指数级权重 (如 $N_{self} = 4$ 赋权 10^6)。

若窗口为污染状态 (即 $N_{self} > 0 \cap N_{op} > 0$), 说明该维度的空间被阻断, 博弈价值归零。

(2) 引入动机与必要性

早期博弈模型常采用“连续字符串匹配算法” (如正则匹配 01110 定义活三)。此类算法具有极强的拓扑脆弱性, 当棋型出现离散化变异 (如跳三/跳四 1011, 1101) 时, 严格的字符串匹配极易发生特征遗漏。

(3) 算法收益

视窗滑动机制摒弃了局部棋子排列的有序性要求, 直接量化连续 5 维绝对空间内的“有效棋子密度”。它将复杂的二维盘面模式识别, 转换为可被 CPU 高速并行执行的标量累加操作, 解决了断层阵型的漏算问题。

3.1.3. 动态分支收束与状态空间压缩

Alpha-Beta 剪枝的效率呈高度偏态分布, 极度依赖于节点的遍历顺序。若盲目拓展所有 $b \approx 225$ 的节点, 即使存在剪枝亦无法在有限时间内下探。

(1) 算法工作机理

在节点展开前, 系统调用启发式预评估算子 $H(s)$, 对所有合法状态子集进行静态特征打分, 并强制执行降序排列。随后, 施加一个动态截断阈值 $N_{limit} = f(Depth)$, 只允许排名前 N_{limit} 的高价值节点入栈进行深搜, 其余节点被截断。

该阈值函数呈“漏斗型”非线性衰减分布:

浅层: 维持较宽的边界 ($N_{limit}=12$), 保障搜索视野的广度, 避免因静态启发式误差导致全局最优解在浅层即被丢弃。

深层: 压缩状态拓展空间 ($N_{limit}=3 \sim 5$)。此时状态已进入高度对抗的战术收敛期, 系统减少发散性探索, 聚焦于核心约束集内的攻防。

(2) 引入动机

若采用静态定长截断 (如恒定截取 Top 10), 由于算力均匀摊派, 会导致系统因算力耗尽只

能推演极浅的层数，无法跨越局部的次优极值，从而跌入人类在深层设置的连环陷阱。

(3) 效能增益

实现了搜索域的指数坍缩。假设向深层拓展 4 个周期，无干预的计算节点量级为 10^4 。经由非线性的 12-8-5-3 动态漏斗截断，理论末端节点数坍缩至 $12 \times 8 \times 5 \times 3 = 1,440$ 个。该机制在维持防漏算阈值的前提下，剥离了超过 85% 的冗余状态空间，使得纯解释型语言亦能突破算力壁垒，扩展更深的搜索域。

3.2. MTD(f)算法

MTD 算法主要组件为 Negmax 算法、零宽探测与迭代加深、以及 Zobrist 分布式哈希表。

3.2.1. Negmax 算法

MTD(f) 对标准 Alpha-Beta 框架进行了改造。算法首先利用零和特性方程 $\max(a,b) = -\min(-a,-b)$ ，通过 Negamax 架构将树的极大极小层合并为单一同构层。

3.2.2. 零宽探测与迭代加深

(1) 算法工作机理

零宽探测：

MTD(f) 放弃了使用宽区间 $[\alpha, \beta]$ 求解精确最优化值的传统思路，转而采用零宽判定区间 $[f-1, f]$ 。在该约束下，搜索树的目标函数由“求解连续域的精确值”退化为“执行离散布尔校验”：即验证当前分支的上限是否超越预期值 f 。一旦子节点的价值发生微小偏移，将触发超高频的 Beta 截断。最后，外部循环通过二分逼近逐步收敛至真实的博弈价值。

迭代加深：

系统不直接挂载最大搜索深度，而是遵循 $Depth \in \{1, 2, 3, \dots, D_{max}\}$ 的递增序列，循环拉起 MTD(f) 探测器。

(2) 增益

鉴于 $O(b^d)$ 的非线性增长特征，浅层 $(1 \dots d-1)$ 遍历的算力总和远小于深层 (d) 的扩展开销 ($<10\%$)。该机制确保在严格的步态限时耗尽 (Timeout) 时，系统可即时阻断进程，并安全返回上一深度的完整闭包决策，尽量将 Depth 推至更深层。

MTD(f) 的效率强烈依赖于初始预期值 f 的偏离度。迭代加深完美解决了最优起点的选取：将第 $n-1$ 层提取的精确静态值注入为第 n 层 MTD(f) 的零宽基准 f 。该机制使得深层搜索的初次布尔探测具备极高置信度，大幅降低了重搜损耗。

3.2.3. Zobrist 分布式哈希与状态复用

在博弈状态转移图中，离散序列的排列组合往往会高频收敛至同一盘面结构。

(1) 算法工作机理

系统初始化阶段，所有空格构建为 64 位的 0。然后利用伪随机数生成器构建一组 $15 \times 15 \times 2$ 的 64 位无符号整数。每次状态转移（落子/撤销），仅需提取对应坐标的随机数，通过单周期指令的按位异或 (XOR) 运算对全局状态量进行更新。该算法生成的哈希签名具备满足均匀分布的极低碰撞率。

系统在 RAM 中开辟置换表 (TT)，映射存储已遍历节点的评估值、最优转移路径及推演深度。遇到重合签名时，系统执行 $O(1)$ 复杂度的查表回溯机制。

(2) 算法收益

利用内存空间置换计算时间。有效阻断了冗余子树的递归重复评估，特别是在存在大量对称及等效演化路径的中局阶段，能够减少 30%~50% 以上的节点扩展开销。

(3) 算法的不足

因为 Python 是解释型语言，存储哈希表所需的内存空间较大 ($\max_depth = 6$ 时，最大开销大约 4G，每步 5-10 秒)。当使用 C++ 等编译型语言（直接生成 CPU 指令）编写本算法时，

内存开销不超过 500M，每步可在 1 秒内执行（ $\text{max_depth} = 8$ ）。这使得我们有机会将 max_depth 上探至 10+（python $\text{max_depth} = 6$ 同等运行时间下，C++ $\text{max_depth} = 12$ ）。

4 算法实现

4.1. Alpha-Beta 剪枝算法

以下将展示各功能模块实现的具体算法。

(1) minimax 算法与 Alpha-Beta 剪枝

```
256 def alphabeta(self, board, depth, alpha, beta, is_maximizing):
272     if is_maximizing:
273         max_val = -math.inf
274         for r, c in candidates:
275             # 试探性落子
276             board[r][c] = self.color
277             # 递归进入下一层（玩家的回合），深度-1
278             val, _ = self.alphabeta(board, depth-1, alpha, beta, False)
279             # 回溯：撤销试探性落子，恢复棋盘
280             board[r][c] = EMPTY
281
282             # 如果这个分支结果比当前已知最好结果还要好，更新最优解
283             if val > max_val:
284                 max_val, best_move = val, (r, c)
285
286             # 更新当前Max层保底的下界 Alpha
287             alpha = max(alpha, val)
288
289             # Beta 剪枝：如果当前发现的下界 Alpha 已经超过了上层 Min 层能容忍的上界 Beta
290             # 说明上层的玩家绝对不会允许走到这个分支，直接砍掉后续所有无效搜索
291             if beta <= alpha: break
292         return max_val, best_move
293
294     else:
295         min_val = math.inf
296         for r, c in candidates:
297             board[r][c] = self.opponent
298             val, _ = self.alphabeta(board, depth-1, alpha, beta, True)
299             board[r][c] = EMPTY
300
301             if val < min_val:
302                 min_val, best_move = val, (r, c)
303
304             # 更新当前Min层容忍的上界 Beta
305             beta = min(beta, val)
306             # Alpha 剪枝
307             if beta <= alpha: break
308         return min_val, best_move
```

(2) 滑动视窗算法

```
125 def evaluate_point(self, board, r, c):
126     """ ...
131     > if board[r][c] != EMPTY: ...
133
134     val = 0
135     directions = [(1,0), (0,1), (1,1), (1,-1)] # 四个考察方向
136
137     # --- 模块1: 收益评估 ---
138     for dr, dc in directions:
139         # 考察包含落子点 (r, c) 的所有长度为 5 的窗口
140         for i in range(-4, 1):
141             my_c, op_c = 0, 0
142             valid = True
143
144             # 统计当前窗口内双方棋子的数量
145             for j in range(5):
146                 nr, nc = r + (i+j)*dr, c + (i+j)*dc
147                 > if 0 <= nr < BOARD_SIZE and 0 <= nc < BOARD_SIZE: ...
151                 > else: ...
154
155                 > if valid: ...
164
165             # --- 模块2: 预见性惩罚机制 ---
166             # 如果我下在这里，会不会使得旁边的空位变成对手的绝杀点？
167             predictive_penalty = 0
168             board[r][c] = self.color
169
170             for dr, dc in directions:
171                 > for step in [-1, 1]: ...
174
175             board[r][c] = EMPTY # 评估完毕，恢复棋盘原状
176             val -= predictive_penalty # 从总分中扣除风险惩罚
177
178             # --- 模块3: 中心偏好 ---
179             # 减去该点离棋盘中心的曼哈顿距离。在开局局势不明朗时，驱使AI往中心下棋
180             val -= abs(r - BOARD_SIZE//2) + abs(c - BOARD_SIZE//2)
181
182         return val
```

(3) 动态分支收束

```
204 def get_candidates(self, board, depth):
205     """
210     candidates = []
211     has_piece = False
212     for r in range(BOARD_SIZE):
213         for c in range(BOARD_SIZE):
214             if board[r][c] != EMPTY:
215                 has_piece = True
216                 continue
217
218             # 半径缩小剪枝：只搜索距离已有棋子周围 3 格范围内的空位
219             # 远离战区的孤立空位不予考虑，极大节省算力
220             near = False
221             for i in range(max(0, r-3), min(BOARD_SIZE, r+4)): ...
222
223             # 如果这个点在战区内，对它进行粗估打分并加入候选列表
224             if near:
225                 val = self.evaluate_point(board, r, c)
226                 candidates.append((val, r, c))
227
228             # 特殊情况处理：如果是开局第一手，直接下在天元（正中心）
229             if not has_piece: ...
230
231             # 根据刚才粗估的分数，从大到小降序排序（Move Ordering）
232             # 让最高价值的点最先被搜索
233             candidates.sort(key=lambda x: x[0], reverse=True)
234
235             # 动态分支收束（Dynamic Beam Pruning）
236             if depth >= 4: limit = 12
237             elif depth == 3: limit = 8
238             elif depth == 2: limit = 5
239             else: limit = 3
240
241             # 绝杀特权收束：如果首选点分数极高（大于千万），说明已经面临绝杀或被绝杀，
242             # 此时放弃广度搜索，强制收束到前两个点位
243             if candidates and candidates[0][0] >= 1e7: ...
244
245             # 截取 Top N 候选点并返回坐标
246             return [(r, c) for _, r, c in candidates[:limit]]
247
248
249
250
251
252
```

4.2. MTD (f)算法

4.2.1. Negamax 算法

```
206 # Negamax (带置换表的极简 Alpha-Beta)
207 def negamax(self, board, current_hash, depth, alpha, beta, color):
208     alpha_orig = alpha
209     """
210     ...
211
212     best_val = -math.inf
213     best_move = candidates[0]
214
215     for move in candidates:
216         r, c = move
217         board[r][c] = color
218         # O(1) 更新哈希指纹，不需要深拷贝棋盘
219         next_hash = current_hash ^ ZOBRIST_TABLE[r][c][color - 1]
220
221         # Negamax 核心：向下传参时，交换并取反 alpha 和 beta，返回值取反
222         val, _ = self.negamax(board, next_hash, depth - 1, -beta, -alpha, 3 - color)
223         val = -val
224
225         board[r][c] = EMPTY # 回溯
226
227         if val > best_val:
228             best_val = val
229             best_move = move
230
231         alpha = max(alpha, val)
232         if alpha >= beta: break # 零宽剪枝
233
234     # 2. 将计算结果存入全局缓存
235     if best_val <= alpha_orig: flag = TT_UPPERBOUND
236     elif best_val >= beta: flag = TT_LOWERBOUND
237     else: flag = TT_EXACT
238
239     # (score, flag, depth, best_move)
240     self.tt[current_hash] = (best_val, flag, depth, best_move)
241     return best_val, best_move
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
```

4.2.2. 零宽探测

```

263 # MTD(f) 零宽探测
264 def mtdf(self, board, current_hash, first_f, depth, color):
265     g = first_f
266     upperbound = math.inf
267     lowerbound = -math.inf
268     best_move = None
269
270     while lowerbound < upperbound:
271         # 永远只用长度为 1 的窗口 [beta-1, beta] 去探测
272         beta = max(g, lowerbound + 1)
273         g, move = self.negamax(board, current_hash, depth, beta - 1, beta, color)
274
275         if move is not None: best_move = move
276
277         if g < beta:
278             upperbound = g # 猜高了, 调低上界
279         else:
280             lowerbound = g # 猜低了, 调高下界
281
282     return g, best_move

```

4.2.3. 迭代加深

```

286 def iterative_deepening(self, board, current_hash, max_depth, color):
287     """
288     滚雪球式搜索: 从 Depth 1 搜到 Max_Depth。
289     将上一层的精确得分作为下一层 MTD(f) 的初始猜测值 f, 极大提升搜索效率。
290     """
291     f = 0
292     best_move = None
293
294     # 由于我们拥有几十万个节点的庞大 Hash 表,
295     # 在思考期间必须关闭 Python 的垃圾回收, 否则会导致画面严重卡顿冻结!
296     gc.disable()
297
298     for d in range(1, max_depth + 1):
299         f, move = self.mtdf(board, current_hash, f, d, color)
300         if move:
301             best_move = move
302         # 一旦发现绝杀或被绝杀, 不需要再往深搜了, 直接返回
303         if abs(f) >= 10000000:
304             break
305
306     gc.enable() # 思考结束, 恢复垃圾回收
307     return best_move

```

4.2.4. Zobrist 分布式哈希与状态复用

```

38 random.seed(42) # 固定种子, 保证每次运行生成的哈希表一致
39 ZOBRIST_TABLE = [[random.getrandbits(64) for _ in range(2)] for _ in range(BOARD_SIZE)] for _ in range(BOARD_SIZE)]
40 # 置换表 Flag 定义
41 TT_EXACT = 0 # 精确值
42 TT_LOWERBOUND = 1 # 下界 (被 Beta 剪枝)
43 TT_UPPERBOUND = 2 # 上界 (被 Alpha 剪枝)
44 class Agent:
45 > def __init__(self, color): ...
51 > def check_surrender(self, board): ...
73 > def evaluate_board(self, board, current_color): ...
120 > def evaluate_point(self, board, r, c, current_color): ...
168 > def get_candidates(self, board, depth, current_color): ...
199 # Negamax (带置换表的极小 Alpha-Beta)
200 def negamax(self, board, current_hash, depth, alpha, beta, color):
201     alpha_orig = alpha
202     # 1. 查询内存缓存 (Zobrist Hash Lookup)
203     tt_entry = self.tt.get(current_hash)
204 > if tt_entry and tt_entry[2] >= depth: ...
210 score = self.evaluate_board(board, color)
211 > if depth == 0 or abs(score) >= 10000000: ...
213 candidates = self.get_candidates(board, depth, color)
214 if not candidates: return score, None
215
216 # 如果缓存中记录了这个局面曾经的最佳走法, 把它排在最前面(极大提升剪枝率)
217 > if tt_entry and tt_entry[3] in candidates: ...
220
221 best_val = -math.inf
222 best_move = candidates[0]
223
224 > for move in candidates: ...
242
243 # 2. 将计算结果存入全局缓存
244 if best_val <= alpha_orig: flag = TT_UPPERBOUND
245 elif best_val >= beta: flag = TT_LOWERBOUND
246 else: flag = TT_EXACT
247
248 # (score, flag, depth, best_move)
249 self.tt[current_hash] = (best_val, flag, depth, best_move)
250 return best_val, best_move

```

本哈希表的维护主要依赖:

1. 初始化伪随机数生成器与张量

```
random.seed(42)
ZOBRIST_TABLE = [[[random.getrandbits(64) for _ in range(2)] for _
in range(BOARD_SIZE)] for _ in range(BOARD_SIZE)]
```

```
class Agent:
```

```
    def negamax(self, board, current_hash, depth, alpha, beta, color):
```

```
        # 3. 遭遇重合签名时, 触发 O(1) 查表回溯
```

```
        tt_entry = self.tt.get(current_hash)
```

```
        if tt_entry and tt_entry[2] >= depth:
```

```
            # ... (校验 flag 后直接 return 缓存的分数和最佳走法)
```

```
        # ... (生成候选点等操作)
```

```
        for move in candidates:
```

```
            r, c = move
```

```
            board[r][c] = color
```

```
            # 2. 通过单周期指令的按位异或(XOR)运算对全局哈希进行增量更新
```

```
            next_hash = current_hash ^ ZOBRIST_TABLE[r][c][color - 1]
```

```
            # 递归搜索...
```

```
            board[r][c] = EMPTY # 回溯
```

```
        # 4. 将最新计算的节点推演结果映射存储至 RAM 置换表(TT)
```

```
        self.tt[current_hash] = (best_val, flag, depth, best_move)
```

4.2.5. C++的 MTD (f)的实现方式

具体代码主体结构如下:

```
415 // MTD(f) 零宽探测: 借助不断的 Negamax 进行边界逼近, 找出精确值
416 Move mtdf(int board[BOARD_SIZE][BOARD_SIZE], uint64_t current_hash, int& first_f, int depth, int cur_color) {
417     int g = first_f;
418     int upperbound = 1000000000;
419     int lowerbound = -1000000000;
420     Move best_move;
421
422 > while (lowerbound < upperbound) { ...
436     first_f = g;
437     return best_move;
438 }
439
440 // 迭代加深框架: 逐层加深限制搜索
441 Move time_bounded_search(int board[BOARD_SIZE][BOARD_SIZE], uint64_t current_hash, int cur_color) {
442     start_time = steady_clock::now();
443     timeout_flag = false;
444     node_count = 0;
445     int f = 0;
446     Move best_move_overall;
447
448     for (int d = 1; d <= MAX_SEARCH_DEPTH; ++d) {
449         Move move = mtdf(board, current_hash, f, d, cur_color);
450         if (timeout_flag) break;
451
452         best_move_overall = move;
453         reached_depth = d;
454
455         if (abs(f) >= 1000000) break;
456     }
457     return best_move_overall;
458 }
459 };
```

值得一提的是, C++模式下的 MTD 算法很容易地将 Max_Depth 扩展至 8, 而所需的 RAM 和运行速度仅为 Python 模式下的数十分之一, 代码申请的内存仅 64M, AI 思考每一步的时间仅 0.x 秒, 相较于 Python 的 5 ~ 10 秒提升数十倍, 在这种情况下, Max_Depth 甚至比

Python 多 2, 意味着计算量扩增近 100 倍, C++实现的 MTD 才能真正发挥 MTD 的效率。
但是取而代之的是较丑的界面。

```
===== GOMOKU AI (C++ MTD(f) Engine) =====
      0 1 2 3 4 5 6 7 8 9 0 1 2 3 4      [COMMANDS MENU]
0 . . . . .
1 . . . . . -> Input 'r c' to Play (e.g. 7 7)
2 . . . . .
3 . . . . . -> Input 'u' to Undo Move
4 . . . . .
5 . . . . . -> Input 'b' to Play Black (Restart)
6 . . . . . -> Input 'w' to Play White (Restart)
7 . . . . .
8 . . . . .
9 . . . . . [Game Status]: Your Turn
0 . . . . . [Last Search Depth]: 0
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
=====

Your move / command: 7 7

===== GOMOKU AI (C++ MTD(f) Engine) =====
      0 1 2 3 4 5 6 7 8 9 0 1 2 3 4      [COMMANDS MENU]
0 . . . . .
1 . . . . . -> Input 'r c' to Play (e.g. 7 7)
2 . . . . .
3 . . . . . -> Input 'u' to Undo Move
4 . . . . .
5 . . . . . -> Input 'b' to Play Black (Restart)
6 . . . . . -> Input 'w' to Play White (Restart)
7 . . . . . X . . . . .
8 . . . . .
9 . . . . . [Game Status]: AI Thinking...
0 . . . . . [Last Search Depth]: 0
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
=====

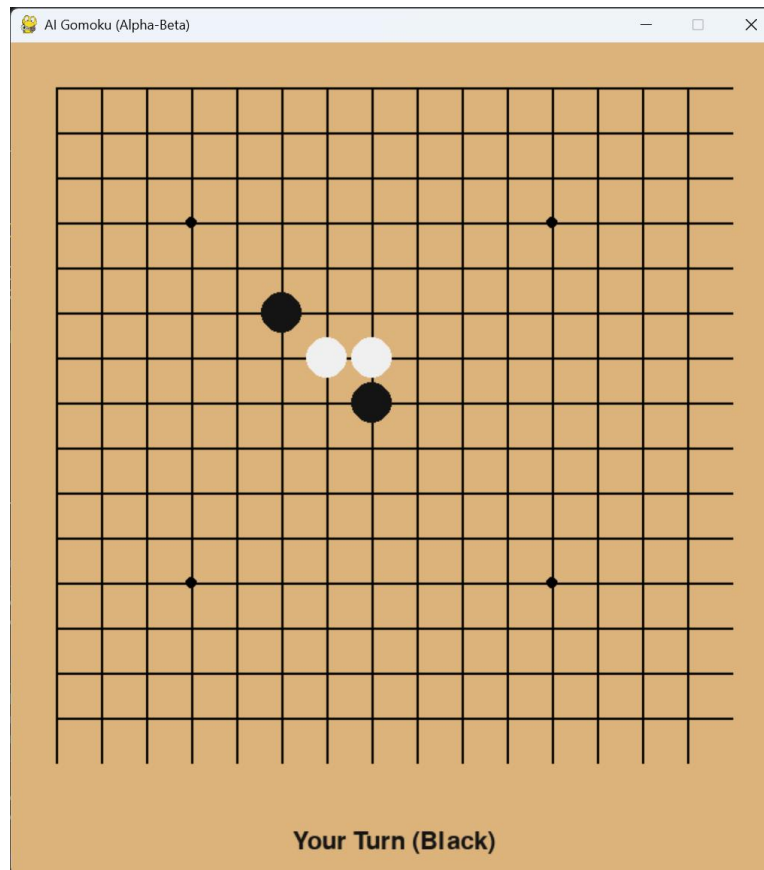
AI is thinking (Depth limit: 8)... Please wait.

===== GOMOKU AI (C++ MTD(f) Engine) =====
      0 1 2 3 4 5 6 7 8 9 0 1 2 3 4      [COMMANDS MENU]
0 . . . . .
1 . . . . . -> Input 'r c' to Play (e.g. 7 7)
2 . . . . .
3 . . . . . -> Input 'u' to Undo Move
4 . . . . .
5 . . . . . -> Input 'b' to Play Black (Restart)
6 . . . . . -> Input 'w' to Play White (Restart)
7 . . . . . O X . . . . .
8 . . . . .
9 . . . . . [Game Status]: Your Turn
0 . . . . . [Last Search Depth]: 8
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
=====
```


5 图形界面美化

以下是针对 Python 代码的图形界面美化。C++ 的界面由于是 cmd，暂无优化方案。

在项目的早期阶段（V1.0 版本），系统 UI 的唯一目标是验证 Alpha-Beta 剪枝代码的正确性。因此，初代界面仅包含一个 15×15 的裸露网格和纯色扁平的圆形棋子，缺乏基本的游戏控制流和视觉反馈。



我做了如下的优化：

5.1. 三层式立体棋子渲染

为了给棋子渲染出立体效果，选择如下方案：

(1) 底层：物理悬浮阴影

将坐标向右下角偏移 (+2, +2)，使用暗灰色绘制稍小的圆

```
pygame.draw.circle(self.screen, (40, 40, 40), (x+2, y+2),  
CELL_SIZE//2 - 2)
```

光学原理：通过偏移的暗色圆，在棋盘底色上方制造出了光线被遮挡的错觉。

(2) 中层：材质底色

```
main_color = (20, 20, 20) if color == BLACK else (240, 240, 240)  
pygame.draw.circle(self.screen, main_color, (x, y), CELL_SIZE//2 - 2)
```

光学原理：覆盖在阴影之上，提供棋子的固有属性颜色。

(3) 顶层：偏心高光点

将坐标向左上角偏移 (-4, -4)，使用极高的亮度，绘制极小半径的圆

```
high_color = (80, 80, 80) if color == BLACK else (255, 255, 255)  
pygame.draw.circle(self.screen, high_color, (x-4, y-4), CELL_SIZE//6)
```

光学原理：模拟位于物理环境左上角的单一光源（如台灯）。光滑的陶瓷或玻璃表面会在光

源直射处产生极亮的高光点。

5.2. 焦点追踪

五子棋到了中局盘面极其密集，人类玩家极易迷失焦点。

技术实现：通过 `self.last_move` 变量记录上一回合落子坐标，利用 `pygame.draw.rect` 在该坐标点绘制一个 10×10 的高亮红色方框。这一红框强制锚定玩家视觉，使其能瞬间捕捉到 AI 或自己的最新动向。

5.3. 终局特效

当初代 AI 认输或游戏结束时，界面极其生硬。

技术实现：引入通道蒙版技术。

```
overlay = pygame.Surface((WIDTH - SIDEBAR_WIDTH, HEIGHT))
overlay.set_alpha(150) # 设置 150/255 的半透明度
overlay.fill((50, 50, 50))
self.screen.blit(overlay, (0, 0))
```

当 AI 触发 `check_surrender` 判定失败时，系统会生成一层带透明度的灰黑色 Surface 覆盖整个博弈区，使得棋盘背景变暗。配合屏幕正中央居中渲染的亮金色大字 YOU WIN，为玩家提供情绪价值。

5.4. 控制台交互功能

初代界面最大的痛点是交互极度受限（如无法悔棋、无法中途重开）。为了解决这一问题，我在最终版的物理视窗上进行了横向扩容：

(1) 界面拓宽与分区：

在原有的 $\text{BOARD_SIZE} \times \text{CELL_SIZE} + \text{MARGIN} \times 2$ 的正方形物理像素坐标系右侧，强制外扩了 $\text{SIDEBAR_WIDTH} = 250$ 像素的控制面板区域。

这使得“博弈战区”与“控制状态区”实现了独立。

(2) 动态侧边栏渲染：

引入了悬停变色的 GUI 按钮组件。通过 `pygame.Rect.collidepoint(mouse_pos)` 实时侦听鼠标轨迹，实现按钮在默认颜色 (`COLOR_BTN`) 与高亮颜色 (`COLOR_BTN_HOVER`) 之间的平滑切换，提供基础的交互阻尼感。

6 总结与反思

本项目以解决零和完全信息博弈中的算力与智力的平衡为核心，进行了从基础 Minimax 框架到 MTD(f) 算法的理论设计与系统实现。在此过程中的工程实践，带来了以下体会与反思：

纯粹依靠堆算力来提升智力的方法是不可持续的。我们永远有越来越复杂的任务，而能掌握的算力却是有限的，不能无限提升，我们必须靠优化算法来获取更高的智力。当下随着大模型的参数逐步增大（据称 Claude Opus 4.6 参数量已达到 5T，前段时间刚发布的 Mythos 更是达到 7.5T - 15.6T，考虑到 MoE 架构，实际激活参数可能在 0.78T 左右），算力需求越来越大，纯粹的力大砖飞路线可能并不可持续。

启发式流形与搜索深度是互相依赖的。搜索树的深度拓展必须建立在精确的启发式流形之上。若静态评估函数存在数学模型上的盲区，更深的搜索不仅无法收敛至全局最优，反而会加速系统坠入局部次优的陷阱。动态分支收束定义了剪枝效率的上限，而评估函数的特征提取能力则决定了系统的认知下限。

软件算法与物理硬件相互制约。在由 Python 向 C++ 转变的过程中，数据打破了“仅凭理

论时间复杂度 $O(b^{\frac{d}{2}})$ 即可衡量系统性能”的刻板印象。Python 中百万级置换表的 GC 扫描

开销,以及 C++ 中 `std::sort` 的递归爆栈和 GCC -O3 循环展开引发的内存激荡,均证明了:抽象的数学算法在落地时,必然受制于物理机的内存连续性、寄存器分配以及指令集行为。尤其是 MTD(f) 在 C++ 中通过连续内存区块(按位与寻址)规避了 Python 拉链法哈希碰撞处理后,运行性能大幅提升,表明面向数据缓存进行底层优化的工程价值。

引入 MTD(f) 是整个系统算力跃迁的转折点。该算法最令人惊叹之处,在于其将“连续域精确极值求解”巧妙转换为“离散布尔校验”的降维思想。通过配合迭代加深提供的校准基准 f , MTD(f) 将零宽判定区间 $[f-1, f]$ 的剪枝威力发挥至极限。这种舍弃单次搜索精度、通过极端约束换取超高频截断、最终由外部循环收敛的博弈思想,对解决高维离散优化问题具有极强的普适性启发。

尽管最终版本的 C++ 在时间内实现 $\text{Depth} \geq 8$,但面对完整五子棋状态空间,该深度仍有局限。未来可尝试引入位棋盘数据结构,以 64 位整数直接映射盘面,利用 CPU 的 SIMD 指令集进行高并发的位运算特征提取。同时,基于 Lazy SMP 架构的多线程并发搜索亦是向双位数搜索深度突破的必由之路。